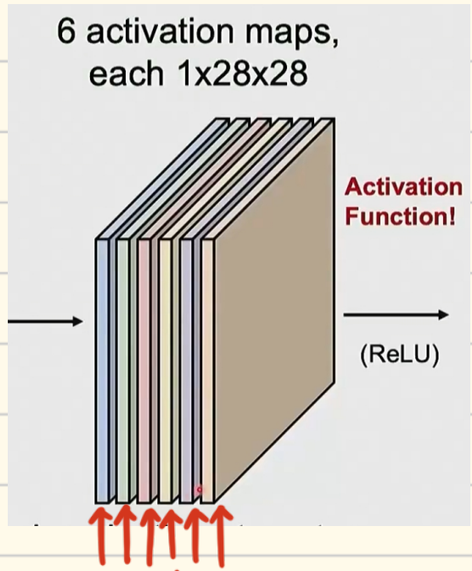
Lecture 6 Training CNNs and CNN Architectures.

recap. 卷积层. 池化层. 全连接层.

↳ 降维.

- Normalization Layers 标准化层.
- Dropout
- Activation Fuctions.



6个channel.

① Normalization Layers. 标准化/归一化层.

顶层思想："移动"、"扩展" 输入.

step: 1. 标准化数据(利用均值.方差等).

2. 移动/扩展.

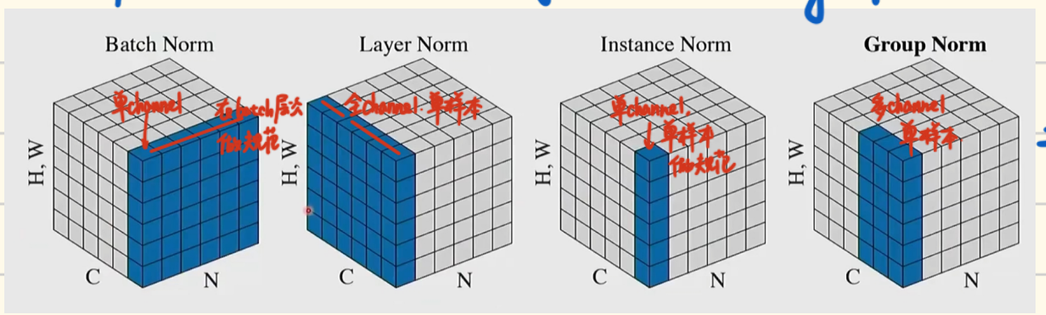
$x: N \times D$ —输入→ μ, σ ($N \times 1$)

γ, β: 可学习的参数. ($1 \times D$). (通道等)

$\longrightarrow y = \gamma(x-\mu)/\sigma + \beta$

γ: 扩展　β: 移动.

不同的规范化: batch. layer, instance, "group"



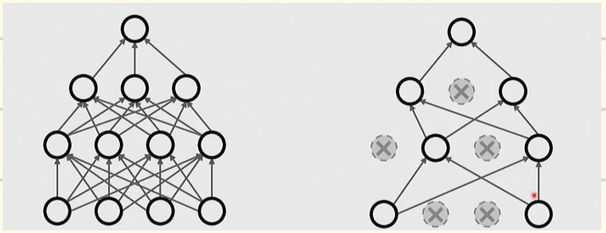
⇒ 都试图规范化数据.

但问题的切入点不同

"channel": 单个"点"的"深度"方向.

② Dropout. 一种正则化层.

基本思想：添加"随机化"的节点连接.
是"正则化"的一种形式



"向前传播"过程中.
随机归零一些输出.

"向后传播"时，由于断连，此处梯度为0.
(就当不存在).
所以不会有变化.

主要超参数：P 删除节点的概率.
(常见 0.5, 0.25).

迫使网络有"冗余"的表示：让模型不再强烈依赖于某些特征，
而是多个特征.

作用范围：仅限训练阶段。测试阶段不会掩盖任何激活。

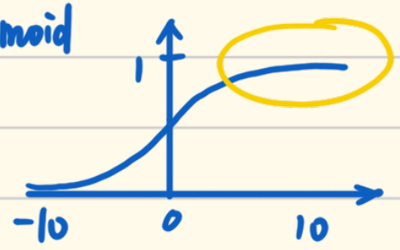
```
def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

注意：由于在测试阶段 dropout 失效，故对层来说
其输入增加了 $\frac{1}{p}$ 倍。需乘 p 使其保持正常.
(更多概率输入)

③ Activation Functions.

总体意义：引入非线性.

Sigmoid



历史上常用的激活函数

主要问题：

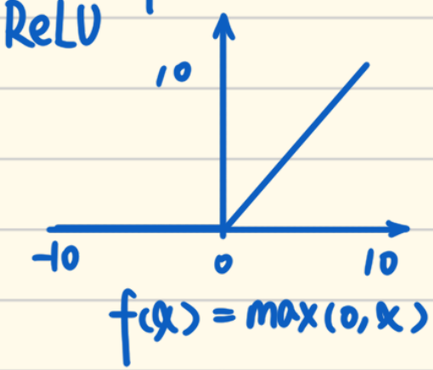
经过多层后对梯度的影响变小.

$\sigma(x) = 1/(1+e^{-x})$

在足够大/小的区域，其梯度以近于0

→ 进入 Sigmoid 的值非常大/小，梯度就非常小.

ReLU (Rectified Linear Unit).



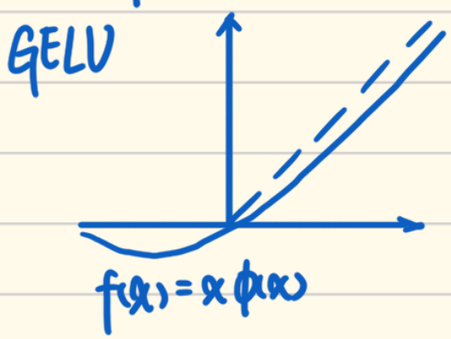
$f(x) = \max(0, x)$

{ 正区域永远有梯度，不会"饱和"
计算非常快

$x<0$ 时，梯度变为0。

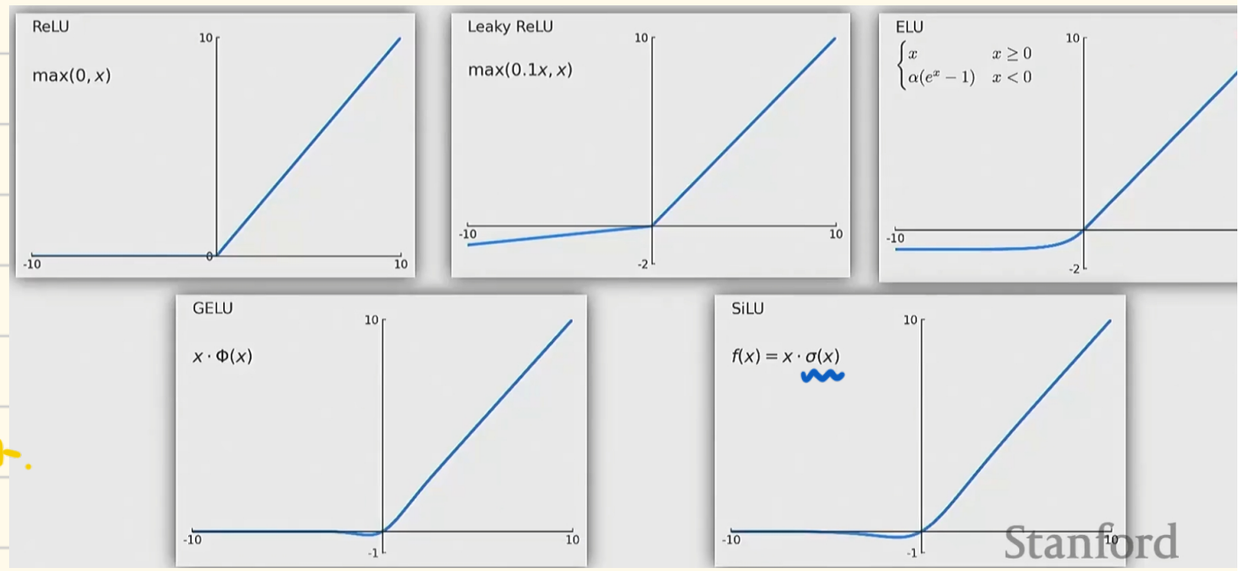
相比 Sigmoid，过大/小都变0。

GELU



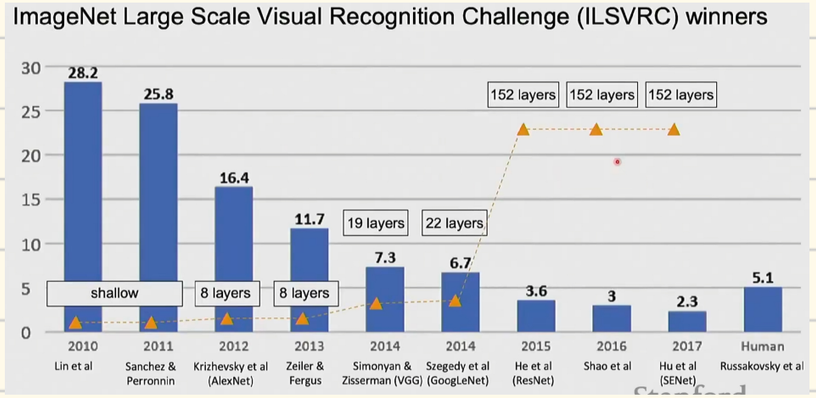
平滑0附近的函数，$-\infty/+\infty$ 仍然为0.

transformer 中常用



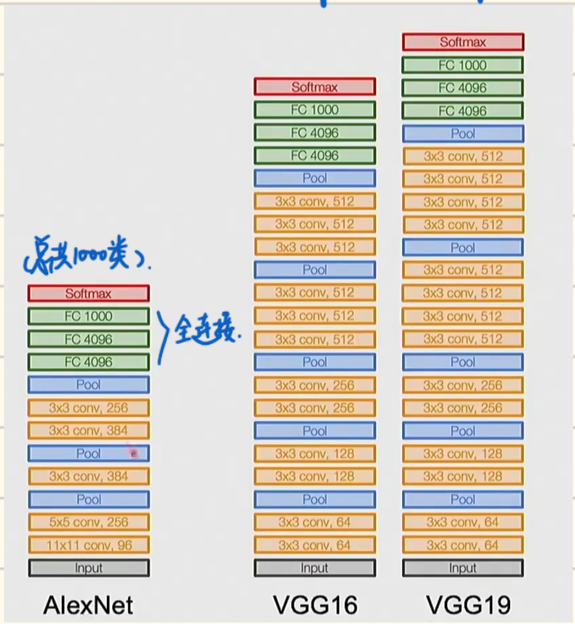
激活函数位置：线性算子之后.（卷积/全连接层）.

(二). CNN架构.



模型错误率 ↓. layers ↑

① AlexNet: 首个CNN架构 ⟶ VGG. 2010后的架构



(block diagram. 框图).

为什么选择3×3，步长1的卷积核？

* 有效感受野：（两个是7×7）

{ input → A1 → A2 → A3
  7×7 ← 5×5 ← 3×3 ← 1×1

好处1：每层都让有效感受野+2

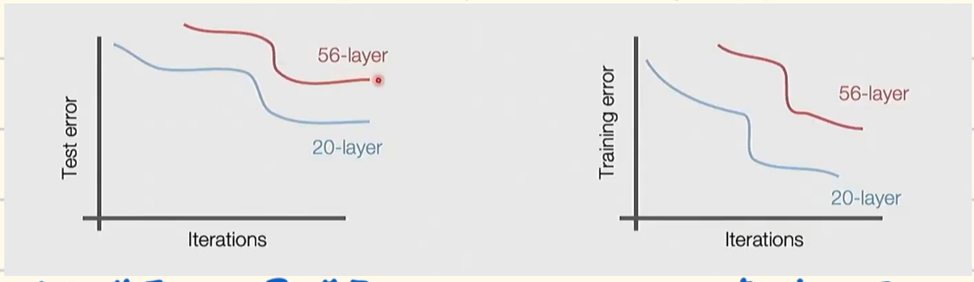
好处2：7×7对比(3×3)×3卷积核.

3×3×3架构的参数更少

对于大卷积核，用小核拼效果更好（有额外ReLU），且参数更少.

② After 2015："层数革命". ResNet

idea：在简单的层堆叠后，模型效果如何？

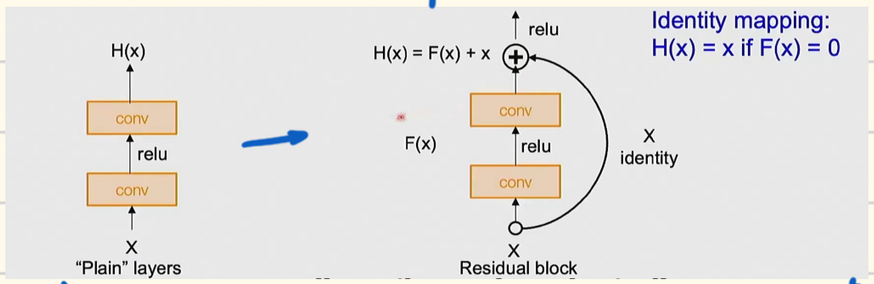


对比56层模型，20层模型在test / training中都表现更好.

↳ 原因：{ ① 不是过拟合。从直觉上，大模型是小模型的超集，理应表现更好.
② 本质是优化问题：小模型更快收敛.
大模型被局部最优卡住.

一个简单的方法：

拟合残差映射.

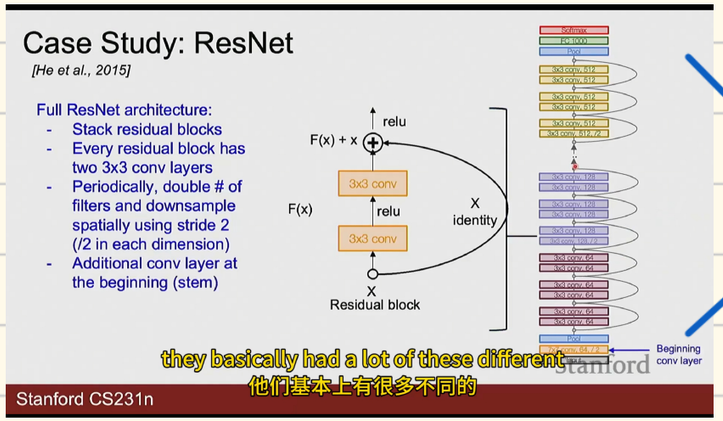


解决方式：将H(x)改为F(x)+x. F(x)代表中间层.

此时在初始将F(x)设为0或极小值，它就"忽略"了这一层.

这样"至少"和小模型一样好.

（避免局部最优，其机理难以具体解析）



depoh 18→34→50→101→152

→ 性能提升至152后无显著提升.

{ 2个3×3的卷积层堆出一个残差层.
残差层再堆出整个模型.

{ 周期性使用步长2来扩展filter的视野.
并且使filter数量翻倍

→ 添加一个更大的7×7层在开头.

步长1：线性提升视野

步长2：2"提升视野，非常快.

(三) Weight 初始化

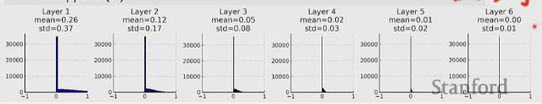
Weight Initialization Case: Values too small

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout) # Small weight init
    x = np.maximum(0, x.dot(W))  # ReLU activation
    hs.append(x)
```

All activations tend to zero for deeper network layers

0.01权重数

又值收敛于0



Stanford

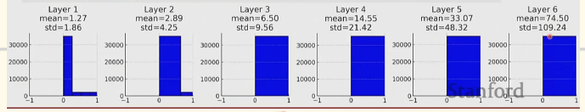
Weight Initialization Case: Values too large

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout) # Small weight init
    x = np.maximum(0, x.dot(W))  # ReLU activation
    hs.append(x)
```

Activations blow up quickly

Increase std of initial weights from 0.01 to 0.05

0.05权重数



Stanford

层间权输出值极剧上升.

(Din:输入维度
Dout:输出维度).
randn.高斯随机.
标准正态分布.

难以确定W系数大小以保证收敛
如何普遍性地初始化?

Kai Ming: 系数取 $\sqrt{2/Din}$ (Din:输入维度).

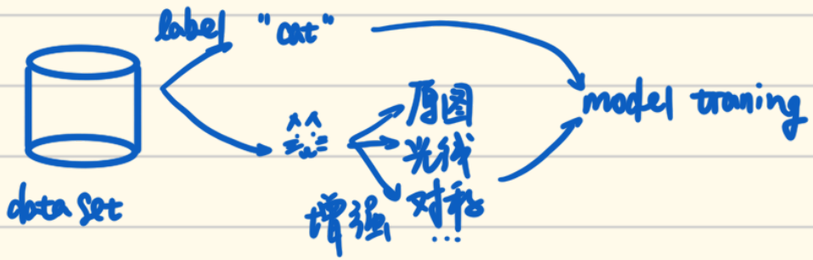
Part 2 : How to train CNNs.

① Data preprocessing.

对图像来说：求出每个颜色的平均值、标准差，
给每个像素每个通道归一化。

$$norm_pixel[i,j,c] = (pixel[i,j,c] - np.mean(pixel[:,:,c])) / np.std(pixel[:,:,c])$$

② Data augmentation 数据增强.



训练：多种"增强"增加数据集

预测：输入图片的多种"增强"取平均.

对称（水平）：对图像、日常物品识别
调整大小 / 裁剪
颜色抖动
Cutout.（例如遮挡环境）.

③ Transfer Learning

小数据集训练：
- 仅训练最后一层，前面用别人的 frozen model.
- 在数据较大时，也可用别人的参数预加载.

> 局限性：别人参数的训练集很可能与你的有差异，却在训练中会产生"偏见".

	相似	不相似
少数据	仅训练靠后一层类"线性分类器"的训练模式.	很遗憾，要么找新数据，要么找相似模型。此情况最难训练.
多数据	增加载参数，全层训练.	视情况而定，有时增加载优秀，有时随机初始化优秀.

④超参数选择：过拟合小样品，检查bugs.

↓尝试不同参数，并作图检查.(在小样本上).

超参数的随机搜索比网格搜索更加有效.